

ARCANJO

Explicacao completa das tecnologias usadas no projeto

Documento tecnico e objetivo para apresentacao academica, defesa do projeto e documentacao do sistema.

Front-end	Back-end	Banco e Infraestrutura
React, Next.js, React DOM, HTML, CSS, JavaScript, PostCSS	Node.js, Express, TypeORM, JWT, bcrypt, CORS, dotenv, Zod, Helmet, Compression	PostgreSQL, pg, Render, GitHub Pages, Docker, Docker Compose, GitHub Actions

Resumo principal: cada tecnologia foi escolhida para separar responsabilidades, melhorar seguranca, organizar o codigo, facilitar manutencao e permitir deploy do sistema.

1. Visao geral da arquitetura

Observacao: este PDF foi atualizado com base no que ja foi discutido sobre o Arcanjo e nos arquivos publicos do repositorio consultados em 30/06/2026, incluindo README, package.json, Dockerfile, docker-compose.yml, render.yaml e workflows do GitHub Actions.

O Arcanjo usa uma arquitetura web separada em camadas. Essa separacao evita misturar interface, regras de negocio, autenticacao e banco de dados no mesmo lugar.

Usuario Navega, publica e interage	Front-end Next.js + React + CSS	API / Back-end Node.js + Express + seguranca	Banco de dados PostgreSQL + TypeORM	Deploy / CI GitHub Pages + Render + Actions
---------------------------------------	------------------------------------	--	---	---

Fluxo resumido: o usuario acessa o front-end, o front-end chama a API, a API valida regras e autenticacao, o TypeORM acessa o PostgreSQL, e o deploy deixa cada parte online.

Resumo das tecnologias por area

Area	Tecnologias	Papel no Arcanjo
Interface	React, Next.js, React DOM, HTML, CSS, JavaScript, PostCSS	Construir telas modernas, responsivas e reutilizaveis.
Servidor/API	Node.js, Express, ES Modules	Criar a API que recebe requisicoes, processa regras e responde ao front-end.
Dados	PostgreSQL, TypeORM, pg, reflect-metadata	Guardar e consultar usuarios, projetos, comentarios, curtidas e demais informacoes.
Seguranca	JWT, bcrypt, CORS, Helmet, Rate Limit, dotenv	Proteger login, senha, rotas, cabecalhos, variaveis sensiveis e acesso indevido.
Validacao e qualidade	Zod, ESLint, Prettier, Vitest, Supertest	Evitar dados invalidos, padronizar codigo e testar a API.
Desenvolvimento e deploy	Nodemon, Docker, Docker Compose, GitHub Actions, Render, GitHub Pages, npm	Facilitar execucao local, automacao, publicacao e manutencao do sistema.

2. Tecnologias do front-end

Tecnologia	O que faz	Por que foi usada no Arcanjo
React	Construcao de interface por componentes.	Foi usado para dividir telas como feed, dashboard, cards, menus, perfil e formularios em partes reutilizaveis. Isso facilita manutencao e evolucao da interface.
Next.js	Framework em cima do React.	Foi usado para organizar paginas, rotas, build, configuracao de basePath para GitHub Pages e exportacao estatica quando necessario. Ajuda a transformar o React em uma aplicacao mais estruturada.
React DOM	Ligacao entre React e navegador.	Permite que os componentes React sejam renderizados no DOM da pagina. Sem ele, a interface React nao aparece no navegador.
JavaScript	Linguagem principal do projeto.	Foi usado tanto no front-end quanto no back-end, deixando o projeto mais padronizado e facilitando o desenvolvimento full-stack.
HTML	Estrutura semantica das paginas.	Serve como base da interface exibida ao usuario: titulos, secoes, formularios, botoes, cards e conteudo.
CSS	Estilizacao visual.	Foi usado para responsividade, cores, espacamentos, layout dos cards, aparencia do feed e identidade visual da rede social.
PostCSS	Processamento de CSS no build.	Aparece nas dependencias do front-end para auxiliar o pipeline de estilos usado pelo Next.js.
Next.js output export	Geracao de site estatico.	Permite gerar a pasta de saida do front-end para hospedagem em GitHub Pages, quando o front precisa ser publicado como arquivos estaticos.
BasePath e assetPrefix	Ajuste de caminho para GitHub Pages.	Como o site pode ficar em /Arcanjo/, esses recursos ajudam imagens, rotas e arquivos estaticos a carregarem

corretamente.

Resumo: React cria a interface; Next.js organiza a aplicacao; HTML, CSS e JavaScript constroem a experiencia visual e interativa.

3. Tecnologias do back-end e da API

Tecnologia	O que faz	Por que foi usada no Arcanjo
Node.js	Ambiente para executar JavaScript no servidor.	Foi usado para rodar o back-end do Arcanjo, permitindo criar uma API usando a mesma linguagem do front-end.
Express	Framework para criar servidor HTTP e rotas.	Organiza endpoints como login, usuarios, projetos, comentarios, busca e health check. Ele recebe requisicoes do front-end e devolve respostas em JSON.
ES Modules	Padrao moderno de import/export do JavaScript.	O package do back-end usa type module, permitindo imports modernos e organizados.
JSON / API REST	Formato de comunicacao entre front e back.	Permite que o Next.js envie e receba dados do back-end de forma simples, como usuario logado, projetos e resultados de busca.
Middleware	Camadas intermediarias nas requisicoes.	Ajuda a aplicar autenticacao, CORS, rate limit, tratamento de erros, validacao e seguranca antes de chegar nas regras principais.
Health check	Rota para verificar se a API esta online.	Usada por plataformas como Render e Docker para saber se o servico esta funcionando corretamente.

Resumo: Node.js roda o servidor; Express organiza a API; middlewares adicionam seguranca, validacao e controle das requisicoes.

4. Banco de dados e acesso aos dados

Tecnologia	O que faz	Por que foi usada no Arcanjo
PostgreSQL	Banco de dados relacional.	Guarda informacoes importantes do Arcanjo, como usuarios, projetos, comentarios, curtidas, salvamentos,

mensagens e configuracoes.

Render PostgreSQL	PostgreSQL hospedado na nuvem.	No deploy atual, o render.yaml cria um banco PostgreSQL chamado arcanjo-db para a API em producao.
Supabase	Hospedagem PostgreSQL com recursos extras.	Foi citado em versoes/ambientes do projeto como opcao para hospedar o PostgreSQL na nuvem, facilitando conexao sem manter um servidor manualmente.
TypeORM	ORM para trabalhar com banco usando objetos.	Evita escrever SQL manualmente o tempo todo. Ajuda a mapear entidades como User, Project e Comment para tabelas no PostgreSQL.
pg	Driver PostgreSQL para Node.js.	E a biblioteca que permite o Node.js conversar diretamente com o PostgreSQL. O TypeORM usa esse driver por baixo.
reflect-metadata	Suporte a metadados usado pelo TypeORM.	Ajuda o TypeORM a interpretar entidades, relacionamentos e configuracoes internas no ambiente JavaScript/TypeScript.
Pool de conexoes	Reaproveitamento de conexoes com o banco.	Evita abrir uma nova conexao a cada requisicao, melhorando desempenho e estabilidade da API.
Seed / RUN_SEED	Carga inicial controlada de dados.	Permite popular o banco em desenvolvimento e evitar seed automatico em producao.

Resumo: PostgreSQL guarda os dados; TypeORM organiza o acesso; pg faz a conexao tecnica entre Node.js e o banco.

5. Autenticacao, seguranca e protecao da API

Tecnologia	O que faz	Por que foi usada no Arcanjo
JWT / jsonwebtoken	Token de autenticacao.	Depois do login, o back-end gera um token que identifica o usuario. O front-end usa esse token para acessar rotas protegidas.
bcrypt	Hash seguro de senhas.	Protege senhas antes de salvar no banco.

		Mesmo que alguém acesse o banco, não vê a senha original do usuário.
CORS	Controle de origem das requisições.	Permite que somente front-ends autorizados acessem a API. Evita que qualquer site externo use o back-end sem permissão.
dotenv	Leitura de variáveis de ambiente.	Mantém segredos e configurações fora do código, como DATABASE_URL, JWT_SECRET, PORT, FRONTEND_URL e chaves de produção.
express-rate-limit	Limitador de requisições.	Reduz abusos, spam e tentativas repetidas de login. Ajuda a proteger a API contra excesso de acessos.
Helmet	Cabecinhos HTTP de segurança.	Adiciona proteções no servidor, reduzindo riscos comuns relacionados a cabecinhos HTTP inseguros.
Compression	Compressão das respostas HTTP.	Diminui o tamanho das respostas enviadas pela API, ajudando no desempenho e no carregamento.
DB_SSL	Conexão segura com banco em produção.	Permite ativar SSL na conexão com banco quando o ambiente de hospedagem exige conexão criptografada.
TRUST_PROXY	Configuração para ambiente com proxy.	Ajuda o Express a funcionar corretamente atrás de plataformas como Render, especialmente para IP real e rate limit.

Resumo: JWT protege o acesso; bcrypt protege senhas; CORS, Helmet, dotenv e rate limit reforçam a segurança da API.

6. Desenvolvimento, padronização e testes

Tecnologia	O que faz	Por que foi usada no Arcanjo
npm	Gerenciador de pacotes do Node.	Instala dependências, executa scripts como dev, build, start, lint, format e test.
package.json	Arquivo de scripts e dependências.	Organiza o que cada parte do projeto precisa para rodar, construir, testar e

		publicar.
package-lock.json	Travamento de versoes.	Garante instalacao mais previsivel das dependencias, principalmente em CI/CD e deploy.
npm ci	Instalacao limpa para automacao.	Usado nos workflows para instalar exatamente o que esta no package-lock, evitando diferencas entre ambientes.
Nodemon	Reinicio automatico no desenvolvimento.	No back-end, reinicia o servidor quando arquivos mudam. Isso acelera testes locais sem precisar parar e rodar o servidor manualmente.
ESLint	Analise de qualidade do codigo.	Aponta erros, problemas de padrao e possiveis bugs antes do deploy.
eslint-config-next	Regras recomendadas para Next.js.	Ajuda o front-end a seguir boas praticas do ecossistema Next e React.
eslint-config-prettier	Compatibilizacao entre ESLint e Prettier.	Evita conflito entre regras de lint e regras de formatacao.
Prettier	Formatador automatico.	Padroniza aspas, ponto e virgula, largura de linha, virgulas finais e indentacao.
Zod	Validacao de dados.	Ajuda a validar entradas recebidas pela API, evitando dados incompletos, invalidos ou inseguros.
Vitest	Framework de testes.	Executa testes automatizados do back-end para garantir que partes importantes continuem funcionando.
Supertest	Teste de rotas HTTP.	Permite testar endpoints da API Express simulando requisicoes sem precisar abrir o sistema manualmente.

Resumo: Nodemon acelera desenvolvimento; ESLint e Prettier padronizam codigo; Vitest e Supertest aumentam confiabilidade.

7. Docker, deploy e automacao

Tecnologia	O que faz	Por que foi usada no Arcanjo
------------	-----------	------------------------------

Docker	Containerizacao da aplicacao.	Empacota back-end e front-end com Node.js, dependencias e comandos de execucao, reduzindo problemas de ambiente.
Dockerfile	Receita para construir uma imagem Docker.	Define como instalar dependencias, copiar arquivos e iniciar backend ou frontend em container.
Docker Compose	Orquestracao local de servicos.	Sobe banco PostgreSQL, back-end e front-end juntos. Facilita testar o sistema completo localmente.
node:22-alpine	Imagem leve do Node.js.	Usada nos Dockerfiles para rodar o projeto com uma imagem menor e mais adequada para producao.
Render	Hospedagem do back-end.	Usado para deixar a API Node.js/Express online, com variaveis de ambiente, health check e banco PostgreSQL.
GitHub Pages	Hospedagem do front-end estatico.	Usado para publicar o build exportado do Next.js como site estatico.
GitHub Actions	Automacao de CI/CD.	Executa instalacao, lint, testes, build e publicacao automaticamente quando ha push na branch main.
CI	Integracao continua.	Verifica qualidade do backend e frontend antes de considerar o codigo pronto.
Deploy workflow	Publicacao automatizada.	Gera o build do frontend e envia para o GitHub Pages automaticamente.
Render Blueprint / render.yaml	Configuracao de infraestrutura como codigo.	Define servico web, banco, comandos de build/start e variaveis de ambiente do deploy.
Health Check	Monitoramento basico de disponibilidade.	Permite que Render e Docker saibam se o backend esta respondendo corretamente.
Git e GitHub	Controle de versao e hospedagem do codigo.	Permitem versionamento, historico de commits, colaboracao, Actions e deploy pelo repositorio.